DNA/RNA Project – Final Report

Group 4

May/June 2025

Supplementary materials (https://github.com/Martinaa1408/DNARNA_Group4/blob/main/DNAmethylation_analysis_manual.pdf)

Pipeline steps

1. Load raw data with minfi and create an object called RGset storing the RGChannelSet object

`minfi` is an R package designed for the in-depth analysis of DNA methylation data obtained through Infinium microarrays. It provides tools for importing raw data, performing quality control, normalization, and a wide range of downstream analyses.
Within the package, the RGChannelSet object represents a data structure that stores the signal intensities from the red and green channels of Illumina microarrays, forming a fundamental basis for quantifying methylation levels across the genome.
`BiocManager` is the official package manager for Bioconductor packages. It automatically selects the Bioconductor repository compatible with our version of R and installs packages from Bioconductor (such as `minfi`), ensuring consistency across releases.

First, check whether the `BiocManager` package is installed; if not, install it.

```
if (!requireNamespace("BiocManager", quietly = TRUE))
  install.packages("BiocManager")
```

Install the `minfi` package using `BiocManager`, if it's not already available.

```
BiocManager::install("minfi")

library(minfi)
library(knitr)
```

Clean the R environment by removing all existing objects and set the working directory, where the raw data are stored.

```
rm(list = ls())
directory = "C:/Users/aurim/Documents/Università/4anno/DNA_RNA_dynamics/Final_Report-2025052
9" #insert here your directory
setwd(directory)
```

Load sample sheet using `read.metharray.sheet()`

```
baseDir <- (directory)
targets <- read.metharray.sheet(baseDir)
```

```
## [read.metharray.sheet] Found the following CSV files:
```

```
## [1] "C:/Users/aurim/Documents/Università/4anno/DNA_RNA_dynamics/Final_Report-20250529/Inpu
t_Data/SampleSheet_Report_II.csv"
```

```
# verify and print the number of imported samples
paste("Number of found samples: ", nrow(targets))
```

```
## [1] "Number of found samples:  8"
```

Read the data using the function `read.metharray.exp` and save it.

```
RGset <- read.metharray.exp(targets = targets)
save(RGset, file = "RGset.RData")
```

2. Create the dataframes Red and Green to store the red and green fluorescences respectively

In this phase of the analysis, extract the red (Cy5) and green (Cy3) fluorescence signals from the RGSet object, which contains the raw data from an Illumina microarray experiment. This is achieved using the `getRed()` and `getGreen()` functions from the `minfi` package, which provide access to the red and green signal channels, respectively.
The extracted signals are converted into two separate data frames, named Red and Green, to facilitate data inspection, manipulation, and analysis.
In each data frame, rows correspond to probes, and columns correspond to samples.

To validate the extraction process, use the `dim()` and `head()` functions. `dim()` checks the dimensions of the matrices, and `head()` displays the first few rows.

```
Red <- data.frame(getRed(RGset))
dim(Red)
head(Red)

Green <- data.frame(getGreen(RGset))
dim(Green)
head(Green)
```

3. Fill the following table: what are the Red and Green fluorescences for the address assigned to your group?
Optional: check in the manifest file if the address corresponds to a Type I or a Type II probe and, in case of Type I probe, report its color.

In this step, we extract fluorescence intensities from the Red and Green channels for the probe identified by address “44666390”.

```
address <- "44666390"
probe_red <- Red[address, ]
probe_green <- Green[address, ]
```

Load the Illumina manifest (*Illumina450Manifest_clean.RData*), in which control probes and SNP-associated probes (those prefixed with “rs”) have been removed to avoid ambiguity due to polymorphisms.

```
load(file.path(directory, "Illumina450Manifest_clean.RData"))
```

Check if the address is in AddressA_ID or AddressB_ID

```
type_A <- Illumina450Manifest_clean[Illumina450Manifest_clean$AddressA_ID == "44666390", 'Inf
inium_Design_Type']
type_B <- Illumina450Manifest_clean[Illumina450Manifest_clean$AddressB_ID == "44666390", 'Inf
inium_Design_Type']
paste("Type from AddressA_ID:",type_A)
```

```
## [1] "Type from AddressA_ID: "
```

```
paste("Type from AddressB_ID:",type_B)
```

```
## [1] "Type from AddressB_ID: I"
```

Determine the actual type

```
actual_type <- ifelse(length(type_A) > 0 && !is.na(type_A), type_A,
                      ifelse(length(type_B) > 0 && !is.na(type_B), type_B, "Not found"))
paste("Type of address:",actual_type)
```

```
## [1] "Type of address: 1"
```

Determine the probe color

```
probe_row <- Illumina450Manifest_clean[
  Illumina450Manifest_clean$AddressA_ID == "44666390" |
  Illumina450Manifest_clean$AddressB_ID == "44666390", ]

actual_color_text <- as.character(probe_row$Color_Channel[1])
if (actual_color_text == "Grn") {
  actual_color_text <- "Green"
}
paste("Color:",actual_color_text)
```

```
## [1] "Color: Red"
```

Create a summary dataframe for the selected address (B), combining the Red and Green fluorescence values for each sample and including the probe type and color.

```
# Final dataframe
df_address <- data.frame(
  Sample = colnames(probe_green),
  Red_fluor = unlist(probe_red, use.names = FALSE),
  Green_fluor = unlist(probe_green, use.names = FALSE),
  Type = actual_type,
  Color = actual_color_text
)

knitr::kable(
  df_address,
  col.names = c("Sample", "Red fluor", "Green fluor", "Type", "Color"),
  align = "lrrcc"
)
```

Sample	Red fluor	Green fluor	Type	Color
GSM5319592_200121140049_R01C02	134	115	1	Red
GSM5319603_3999356129_R02C02	88	66	1	Red
GSM5319604_3999547012_R02C01	215	193	1	Red
GSM5319607_3999547012_R03C02	329	221	1	Red
GSM5319609_3999547016_R02C01	151	104	1	Red
GSM5319613_3999547017_R02C01	256	202	1	Red
GSM5319615_3999547017_R04C01	274	215	1	Red
GSM5319616_3999547017_R06C01	155	91	1	Red

4. Create the object MSet.raw

Convert the RGChannelSet to an MSet object using `preprocessRaw`.
This function takes the Red and the Green channel of an Illumina methylation array, together with its associated manifest object and converts it into a MethylSet containing the methylated and unmethylated signal.

```
BiocManager::install("IlluminaHumanMethylation450kmanifest")
MSet.raw <- preprocessRaw(RGset)
MSet.raw
save(MSet.raw,file="MSet_raw.RData")
```

5. Perform the following quality checks and provide a brief comment to each step:

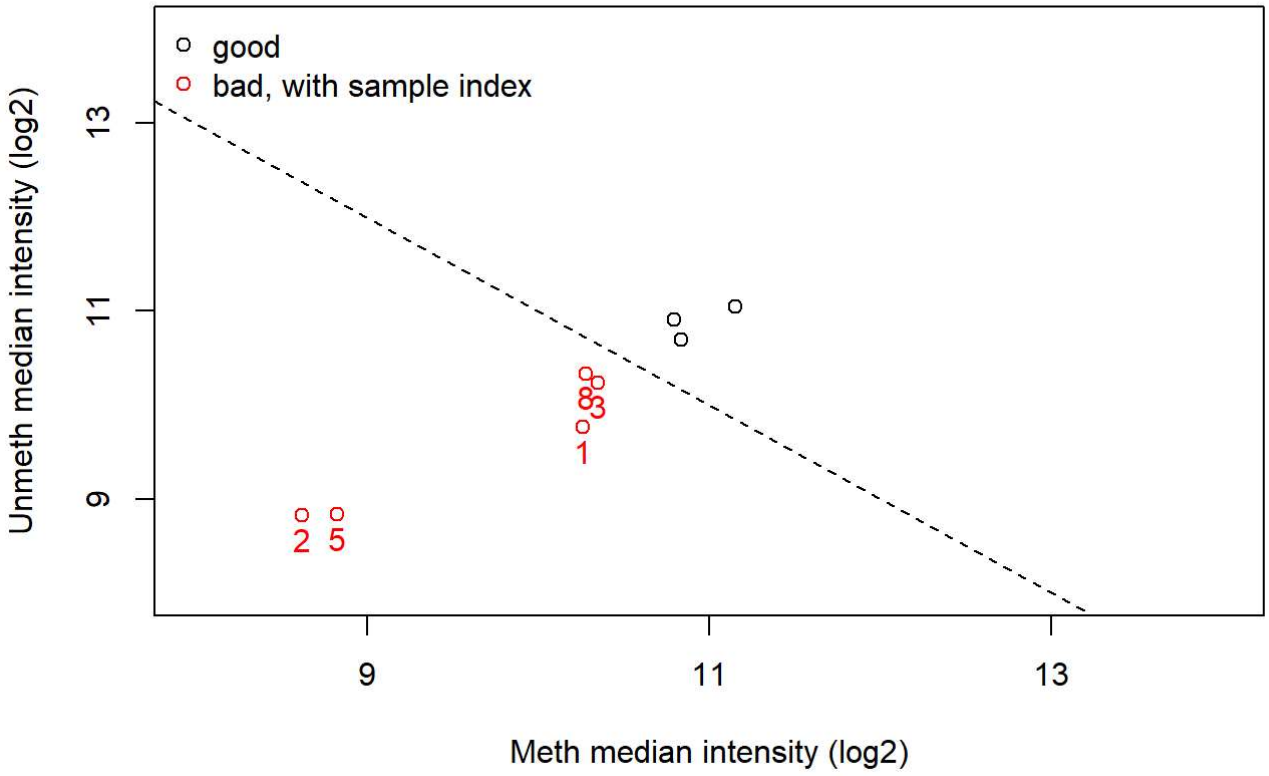
- **QCplot**
- **check the intensity of negative controls using minfi**
- **calculate detection p-values for each sample, how many probes have a detection p-value higher than the threshold of 0.01?**

QCplot

In a QCplot, samples methylation and unmethylation median signals are plotted. Good quality samples are identified in the upper-right section of the graph.

```
qc <- getQC(MSet.raw)
qc
```

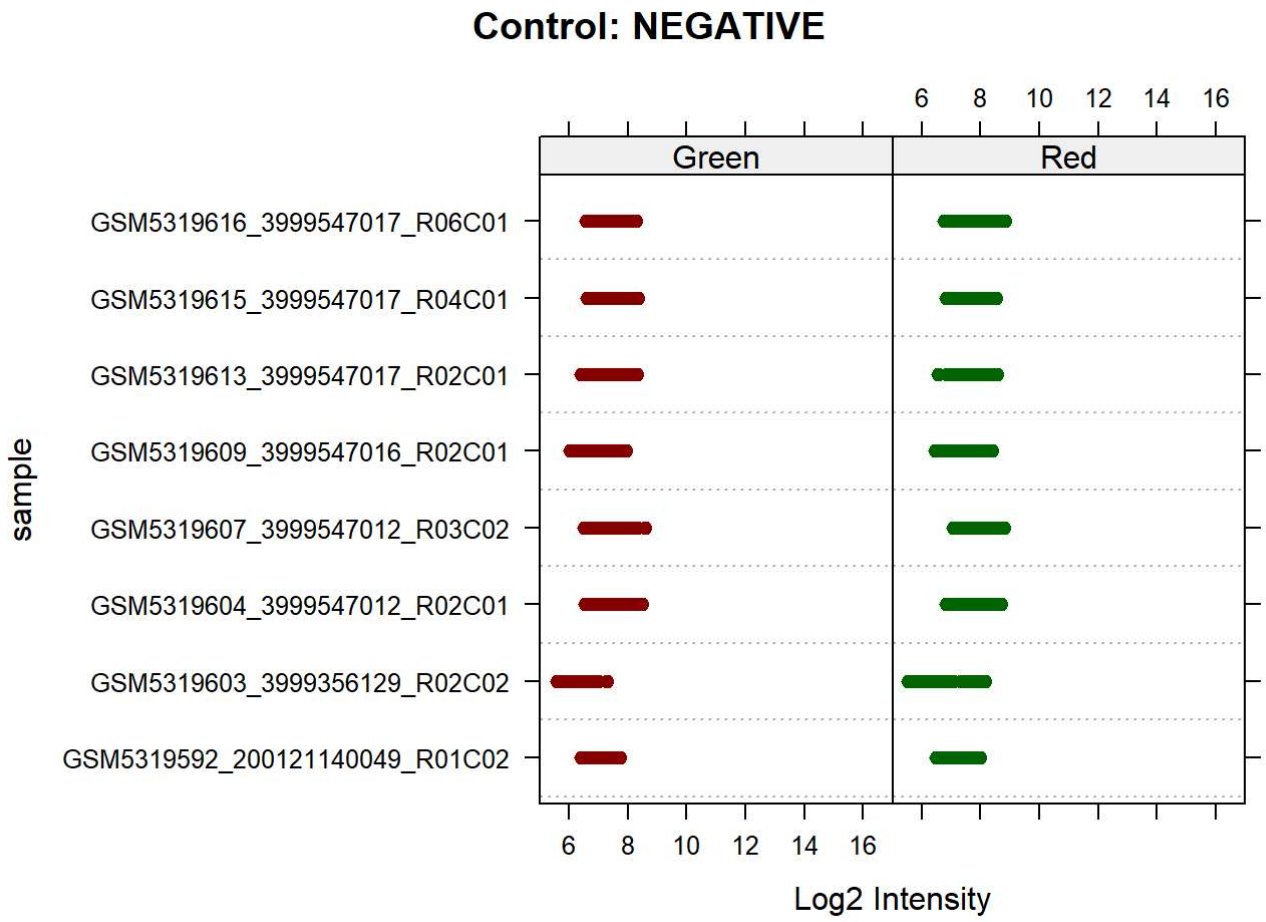
```
plotQC(qc)
```



Samples 1,3,8 are found slightly outside the threshold line, while samples 2 and 5 raise more concerns as they are far from optimal.

Check the intensity of negative controls using `minfi` *.*

```
controlStripPlot(RGset, controls="NEGATIVE")
```



In this plot are reported the intensity levels of the control probes. All sample controls display intensities that are below 10, which is to be expected.

Calculate detection p-values for each sample, how many probes have a detection p-value higher than the threshold of 0.01?

```
threshold <- 0.01
detP <- detectionP(RGset)
failed <- detP> threshold

num_failed = colSums(failed)
df_failed <- data.frame(Sample=colnames(probe_green), Failed_Position=num_failed, perc_failed
_probes=(means_of_columns <- colMeans(failed))*100, row.names = NULL)

knitr::kable(
  df_failed,
  col.names = c("Sample", "Failed Position", "% Failed Probes"),
  align = "lrc",
  digits = 2
)
```

Sample	Failed Position	% Failed Probes
GSM5319592_200121140049_R01C02	303	0.06
GSM5319603_3999356129_R02C02	563	0.12
GSM5319604_3999547012_R02C01	1702	0.35
GSM5319607_3999547012_R03C02	611	0.13

Sample	Failed Position	% Failed Probes
GSM5319609_3999547016_R02C01	4555	0.94
GSM5319613_3999547017_R02C01	573	0.12
GSM5319615_3999547017_R04C01	467	0.10
GSM5319616_3999547017_R06C01	994	0.20

Here are plotted all samples that failed the significance test, as their p-value was over the set threshold (0.01%). This means that the signal captured by these probes cannot be distinguished from noise. From this dataframe is possible to highlight 4 different categories:
- High quality: sample 1 and 7 (less than 0.01% failed probes)
- Good quality: sample 2,4 and 6 (less than 0.2% failed probes)
- Low quality: sample 3 and 8 (above 0.2% failed probes)
- Critical: sample 5 (nearly 1% of failed probes)

Given the results it might be appropriate to exclude sample 5 from the research, since it failed the QCplot and it possesses a considerable amount of failed probes.

6. Calculate raw beta and M values and plot the densities of mean methylation values, dividing the samples in CTRL and DIS (suggestion: subset the beta and M values matrixes in order to retain CTRL or DIS subjects and apply the function mean to the 2 subsets). Do you see any difference between the two groups?

Beta-values and *M-values* are complementary measures of DNA methylation: beta-values quantify the proportion of methylated signal over the total (0–1 range), while M-values express the log2 ratio of methylated to unmethylated intensities, offering better statistical properties for differential analysis.

Select groups (CTRL and DIS) from targets.

```
ctrl <- basename(targets[targets$Group == "CTRL", "Basename"])
dis <- basename(targets[targets$Group == "DIS", "Basename"])
```

Remove any suffixes (e.g., .idat).

```
ctrl <- gsub("\\.idat$", "", ctrl)
dis <- gsub("\\.idat$", "", dis)
```

Print the group samples for verification.

```
cat("CTRL samples:", ctrl, "\n")
```

```
## CTRL samples: GSM5319592_200121140049_R01C02 GSM5319607_3999547012_R03C02 GSM5319615_39995
47017_R04C01 GSM5319616_3999547017_R06C01
```

```
cat("DIS samples:", dis, "\n")
```

```
## DIS samples: GSM5319603_3999356129_R02C02 GSM5319604_3999547012_R02C01 GSM5319609_39995470
16_R02C01 GSM5319613_3999547017_R02C01
```

Subset the MSet object by groups

```
ctrlSet <- MSet.raw[, ctrl]
disSet <- MSet.raw[, dis]
```

Calculate Beta and M values.

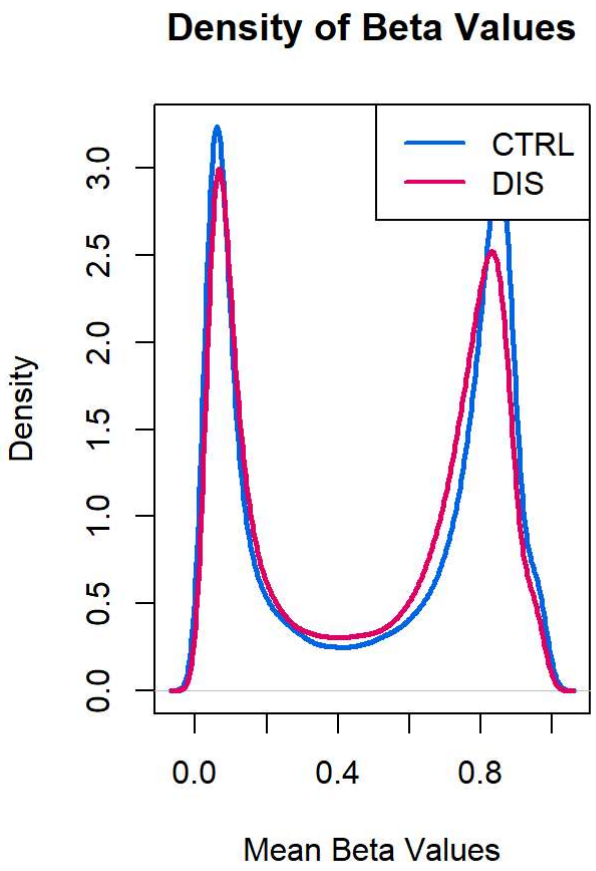
```
ctrlBeta <- getBeta(ctrlSet)
ctrlM <- getM(ctrlSet)
disBeta <- getBeta(disSet)
disM <- getM(disSet)
```

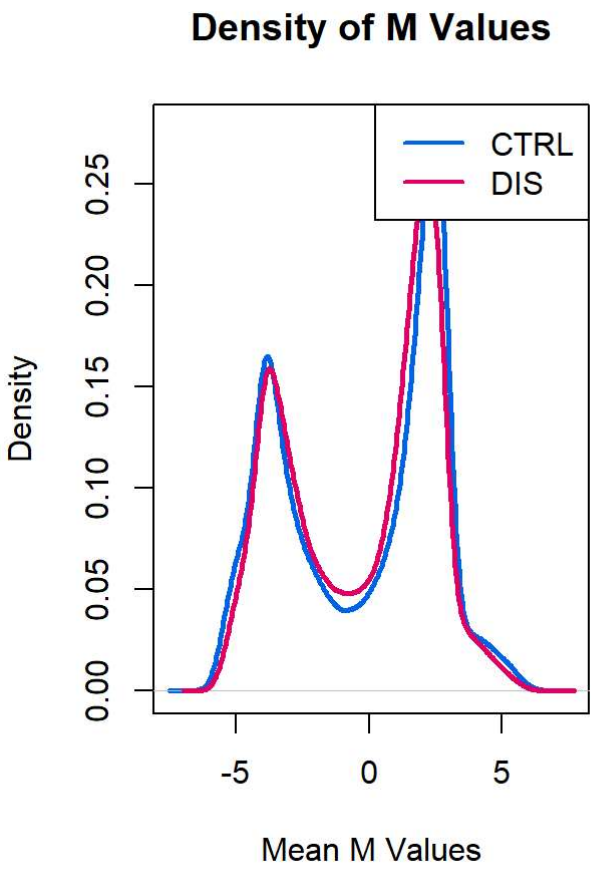
Calculate mean values per probe across samples.

```
mean_ctrlBeta <- rowMeans(ctrlBeta, na.rm = TRUE)
mean_disBeta <- rowMeans(disBeta, na.rm = TRUE)
mean_ctrlM <- rowMeans(ctrlM, na.rm = TRUE)
mean_disM <- rowMeans(disM, na.rm = TRUE)
```

Plot the density of mean Beta and M values.

```
par(mfrow = c(1, 2))

# Density of mean Beta values
plot(density(mean_ctrlBeta, na.rm = TRUE),
     main = "Density of Beta Values",
     col = "#0067E6", lwd = 2.5,
     xlab = "Mean Beta Values", ylab = "Density")
lines(density(mean_disBeta, na.rm = TRUE), col = "#E50068", lwd = 2.5)
legend("topright", legend = c("CTRL", "DIS"), col = c("#0067E6", "#E50068"), lwd = 2)

# Density of mean M values
plot(density(mean_ctrlM, na.rm = TRUE),
     main = "Density of M Values",
     col = "#0067E6", lwd = 2.5,
     xlab = "Mean M Values", ylab = "Density")
lines(density(mean_disM, na.rm = TRUE), col = "#E50068", lwd = 2.5)
legend("topright", legend = c("CTRL", "DIS"), col = c("#0067E6", "#E50068"), lwd = 2)
```






```
#Reset plotting layout
par(mfrow = c(1, 1))
```

Density of Beta values: The graph shows a bimodal distribution, that is to be expected for DNA methylation analysis. The first peak (β ~ 0.1) explains the proportion of unmethylated sites, while the peak on the right (β ~ 0.9) displays methylated sites.

Density of M values: Bimodal symmetric distribution. The two peaks explain the proportion of unmethylated (M ~ -2) and methylated sites(M ~ +2).

There are no clear differences between the control and disease group. This means that globally the methylation level distribution is the same, which is to be expected, as disease associated sites are localized in most cases.

7. Normalize the data using the function assigned to each group and compare raw data and normalized data. Produce a plot with 6 panels in which, for both raw and normalized data, you show the density plots of beta mean values according to the chemistry of the probes, the density plot of beta standard deviation values according to the chemistry of the probes and the boxplot of beta values. Provide a short comment about the changes you observe.
Optional: do you think that the normalization approach that you used is appropriate considering this specific dataset? Try to color the boxplots according to the group (CTRL and DIS) and check whether the distribution of methylation values is different between the two groups, before and after normalization.

Subset the manifest by probe chemistry (Type I and Type II).

```
dfI <- Illumina450Manifest_clean[Illumina450Manifest_clean$Infinium_Design_Type == "I", ]
dfII <- Illumina450Manifest_clean[Illumina450Manifest_clean$Infinium_Design_Type == "II", ]
dfI <- droplevels(dfI)
dfII <- droplevels(dfII)
```

Subset raw beta values by chemistry.

```
beta_raw <- getBeta(MSet.raw)
beta_I_raw <- beta_raw[rownames(beta_raw) %in% dfI$IlmnID, ]
beta_II_raw <- beta_raw[rownames(beta_raw) %in% dfII$IlmnID, ]
```

Calculate mean and sd for raw Beta.

```
mean_beta_I_raw <- rowMeans(beta_I_raw, na.rm = TRUE)
mean_beta_II_raw <- rowMeans(beta_II_raw, na.rm = TRUE)
sd_beta_I_raw <- apply(beta_I_raw, 1, sd, na.rm = TRUE)
sd_beta_II_raw <- apply(beta_II_raw, 1, sd, na.rm = TRUE)
```

Normalize using `preprocessFunnorm`.
Functional normalization (FunNorm) is a between-array normalization method for Illumina 450K arrays that removes unwanted technical variation by regressing out the variability captured by control probes present on the array.

```
BiocManager::install("IlluminaHumanMethylation450kanno.ilmn12.hg19")
MSet.norm <- preprocessFunnorm(RGset)
beta_norm <- getBeta(MSet.norm)
```

Subset normalized Beta by chemistry.

```
beta_I_norm <- beta_norm[rownames(beta_norm) %in% dfI$IlmnID, ]
beta_II_norm <- beta_norm[rownames(beta_norm) %in% dfII$IlmnID, ]
```

Calculate mean and sd for normalized Beta.

```
mean_beta_I_norm <- rowMeans(beta_I_norm, na.rm = TRUE)
mean_beta_II_norm <- rowMeans(beta_II_norm, na.rm = TRUE)
sd_beta_I_norm <- apply(beta_I_norm, 1, sd, na.rm = TRUE)
sd_beta_II_norm <- apply(beta_II_norm, 1, sd, na.rm = TRUE)
```

Density plots for normalized Beta values.

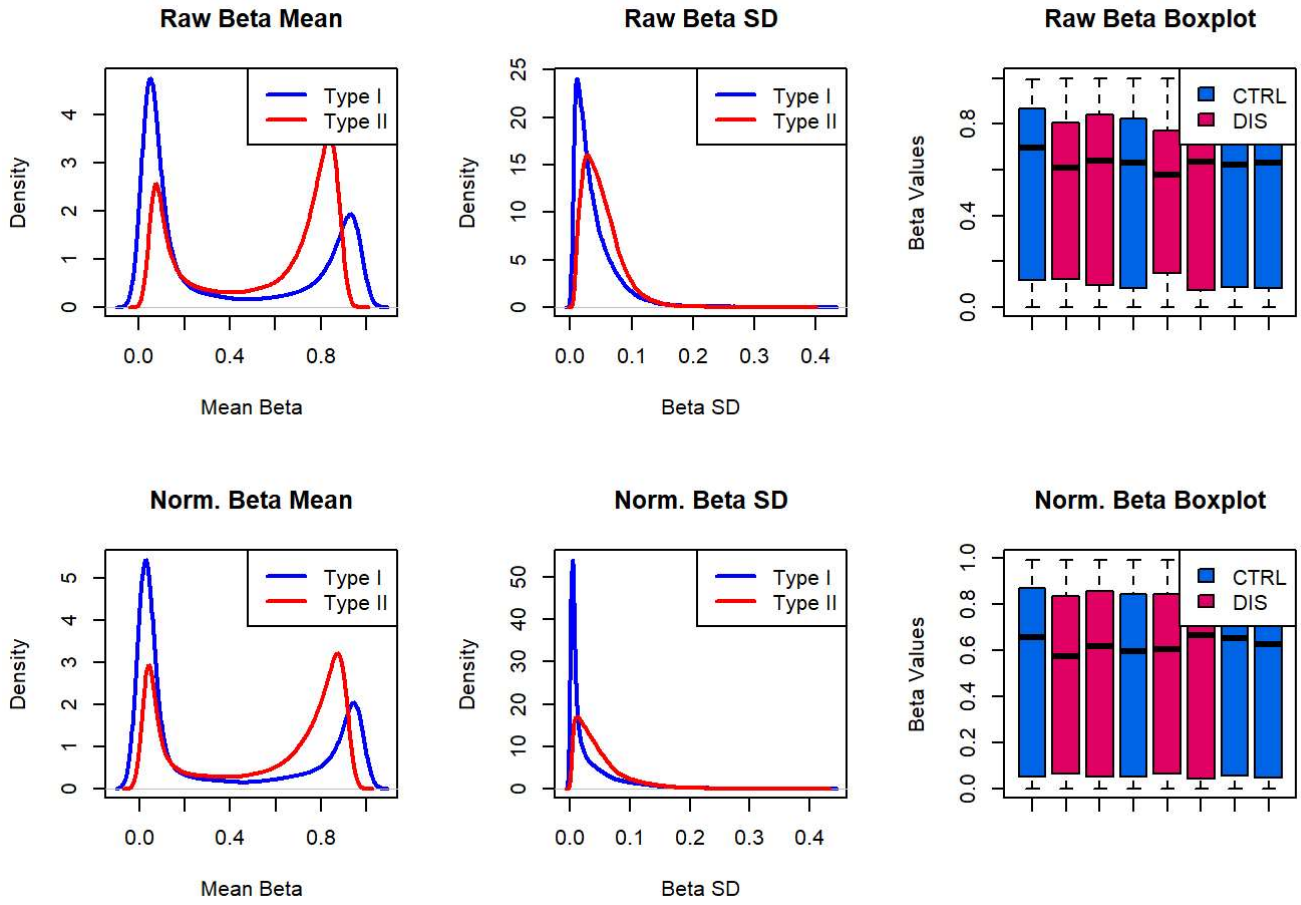
```
density_mean_beta_I_norm <- density(na.omit(mean_beta_I_norm))
density_mean_beta_II_norm <- density(na.omit(mean_beta_II_norm))
density_sd_beta_I_norm <- density(na.omit(sd_beta_I_norm))
density_sd_beta_II_norm <- density(na.omit(sd_beta_II_norm))
```

Prepare the colors for the groups.

```
sample_prefixes <- sapply(strsplit(colnames(beta_raw), "_"), `[`, 1)
group_colors <- ifelse(sample_prefixes %in% targets[targets$Group == "CTRL", "SampleID"], "#0
067E6", "#E50068")
```

Plotting: 2 rows x 3 columns.

```
par(mfrow = c(2, 3))
# RAW DATA (first row)
# 1. Raw mean Beta densities
plot(density(mean_beta_I_raw, na.rm = TRUE), main = "Raw Beta Mean", col = "blue", lwd = 2, x
lab = "Mean Beta")
lines(density(mean_beta_II_raw, na.rm = TRUE), col = "red", lwd = 2)
legend("topright", legend = c("Type I", "Type II"), col = c("blue", "red"), lwd = 2)

# 2. Raw sd Beta densities
plot(density(sd_beta_I_raw, na.rm = TRUE), main = "Raw Beta SD", col = "blue", lwd = 2, xlab
= "Beta SD")
lines(density(sd_beta_II_raw, na.rm = TRUE), col = "red", lwd = 2)
legend("topright", legend = c("Type I", "Type II"), col = c("blue", "red"), lwd = 2)

# 3. Raw Beta boxplot (colored by group)
boxplot(beta_raw, outline = FALSE, col = group_colors,
        main = "Raw Beta Boxplot", ylab = "Beta Values", names = rep("", ncol(beta_raw)))
legend("topright", legend = c("CTRL", "DIS"), fill = c("#0067E6", "#E50068"))

# NORMALIZED DATA (second row)
# 4. Normalized mean Beta densities
plot(density(mean_beta_I_norm, na.rm = TRUE), main = "Norm. Beta Mean", col = "blue", lwd =
2, xlab = "Mean Beta")
lines(density(mean_beta_II_norm, na.rm = TRUE), col = "red", lwd = 2)
legend("topright", legend = c("Type I", "Type II"), col = c("blue", "red"), lwd = 2)

# 5. Normalized sd Beta densities
plot(density(sd_beta_I_norm, na.rm = TRUE), main = "Norm. Beta SD", col = "blue", lwd = 2, xl
ab = "Beta SD")
lines(density(sd_beta_II_norm, na.rm = TRUE), col = "red", lwd = 2)
legend("topright", legend = c("Type I", "Type II"), col = c("blue", "red"), lwd = 2)

# 6. Normalized Beta boxplot (colored by group)
boxplot(beta_norm, outline = FALSE, col = group_colors,
        main = "Norm. Beta Boxplot", ylab = "Beta Values", names = rep("", ncol(beta_norm)))
legend("topright", legend = c("CTRL", "DIS"), fill = c("#0067E6", "#E50068"))
```




```
#Reset plotting layout
par(mfrow = c(1, 1))
```

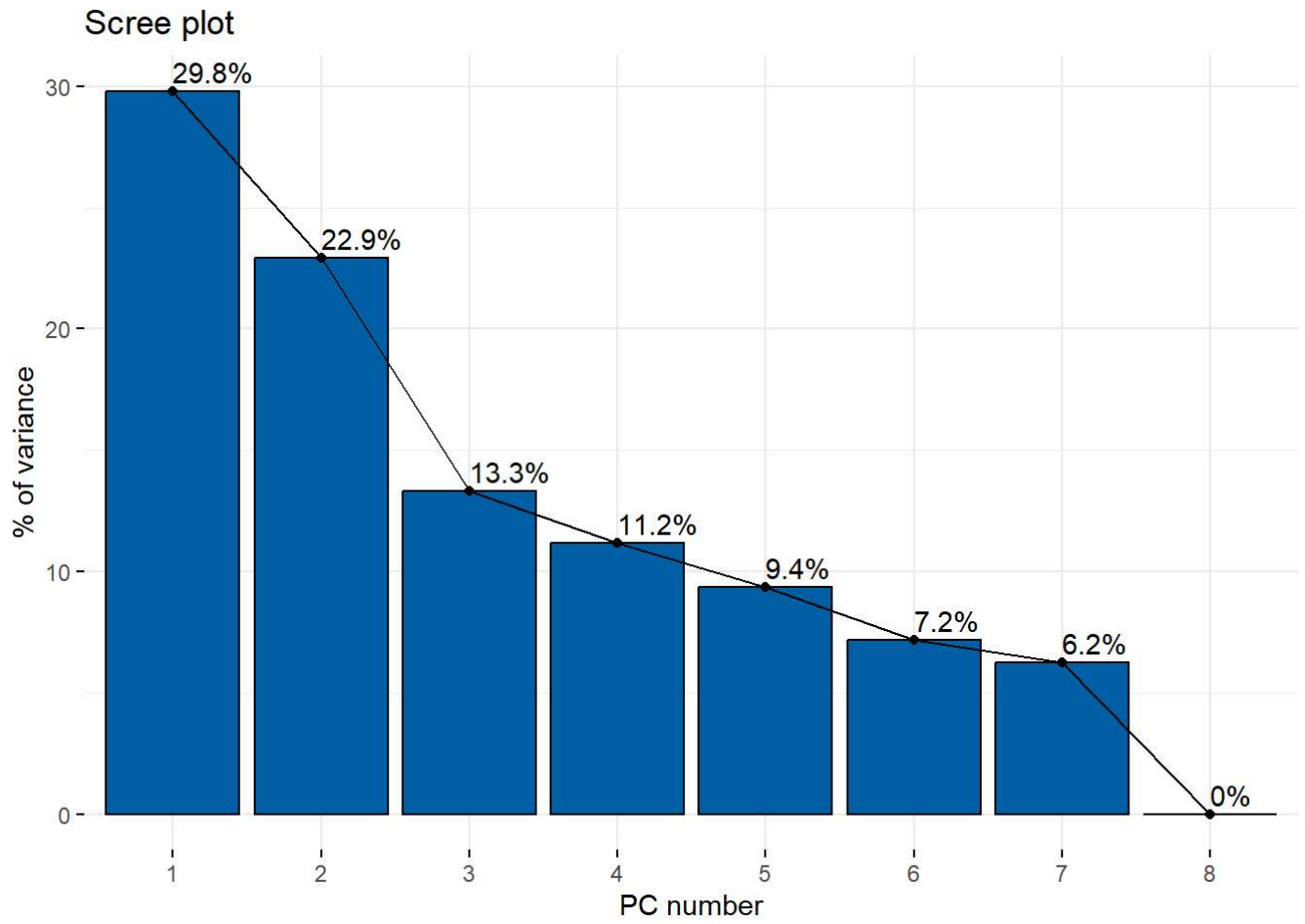
The raw distributions of Beta mean and Beta standard deviation are clearly distinct between Type I and Type II probes. This can be explained by the chemistry implied in each probe type, which is different.
The functional normalization clearly reduced the variability between type I and type II probes, using control intensities to systematically correct data.
The box plot graph results are more uniform, while still preserving sample-specific variability.

8. Perform a PCA on the matrix of normalized beta values generated in step 7, after normalization. Comment the plot (Do you see any outlier?
Do the samples divide according to the group?
Do they divide according to the sex of the samples?
Do they divide according to the batch, that is the column Sentrix_ID?).

Principal Component Analysis (PCA) is an unsupervised technique used to reduce data dimensionality while preserving key variability. It transforms correlated variables into uncorrelated principal components (PCs), allowing the identification of patterns and the main sources of variance.
In this context, PCA is applied to evaluate whether CTRL and DIS samples show clear separation based on their gene expression profiles, potentially indicating biological differences.
A scree plot displays the variance explained by each component and guides the selection of relevant PCs.

Install required packages:
`ggplot2` – implements the grammar of graphics for customizable PCA visualizations.
`ggpubr` – provides simplified functions for generating publication-quality plots. `cluster` – includes clustering algorithms and validation indices
`factoMineR` – performs multivariate methods including PCA.
`factoextra` – extracts and visualizes results from PCA and clustering.

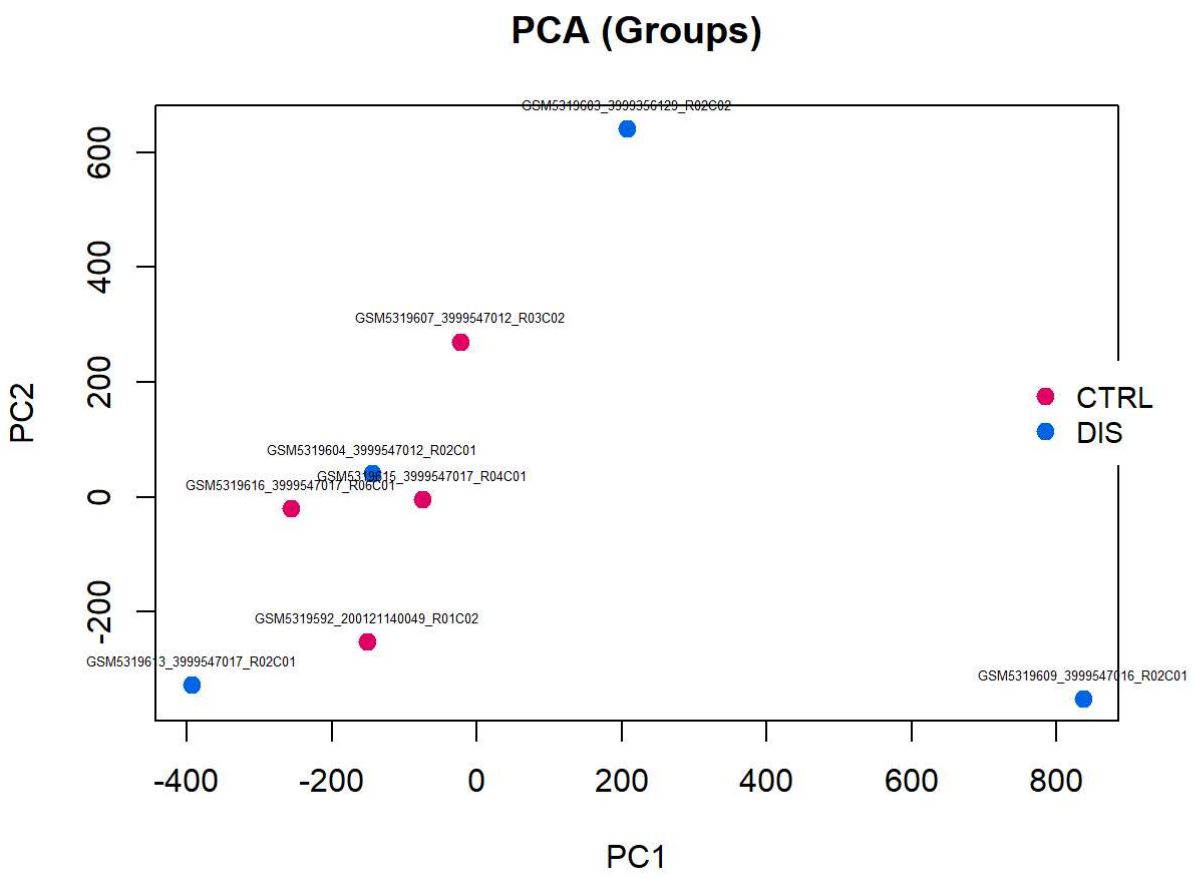
```
pca_results <- prcomp(t(beta_norm), scale = TRUE)
options(repos = c(CRAN = "https://cloud.r-project.org"))
install.packages(c("ggplot2", "ggpubr", "cluster", "factoMineR"))
install.packages("factoextra")
library(factoextra)
fviz_eig(pca_results, addlabels = TRUE, xlab = 'PC number', ylab = '% of variance', barfill =
"#0063A6", barcolor = "black")
```



The scree plot shows that PC1 (29.8%) and PC2 (22.9%) together explain over 50% of the total variance. This means that the first two principal components capture the main structure in the normalized beta matrix, and are suitable for interpreting the most relevant trends in the data.

PCA Plot by Group

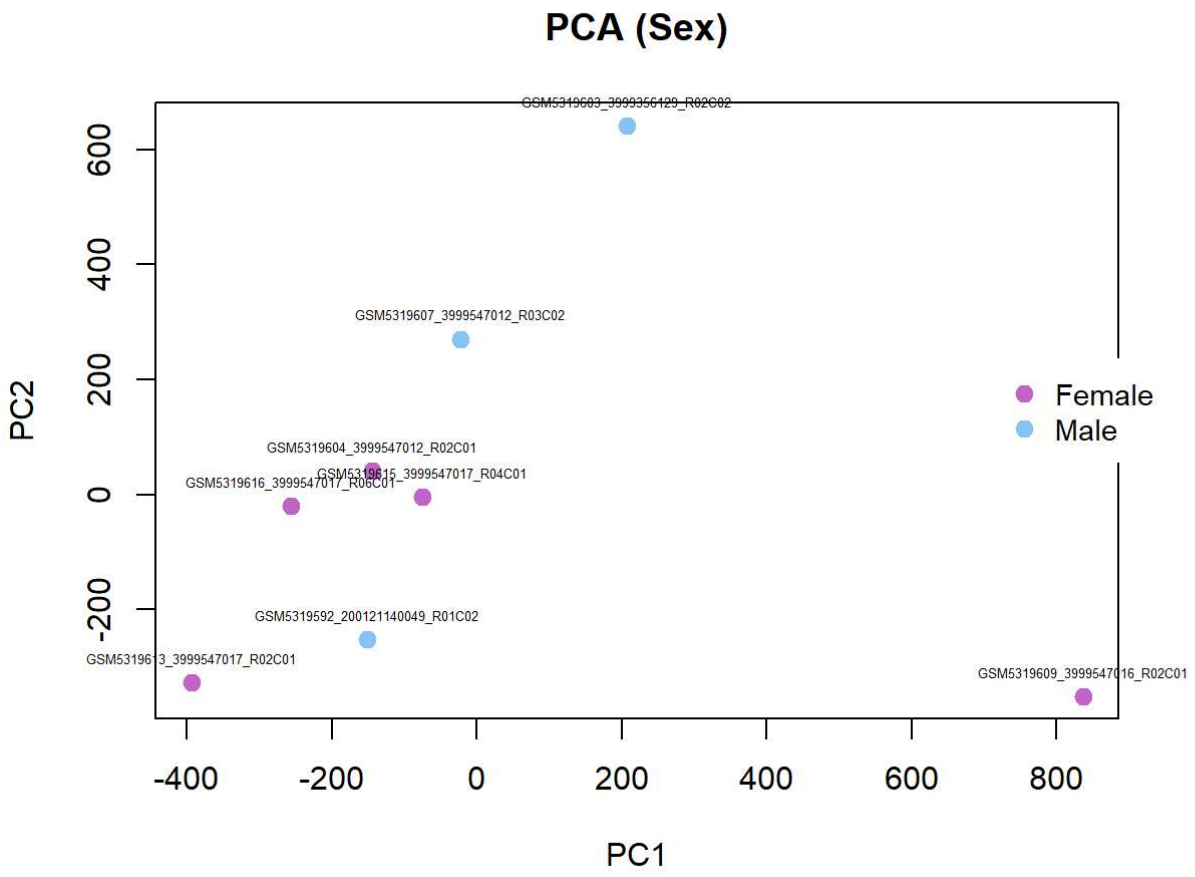
```
targets$Group <- as.factor(targets$Group)
palette(c("#E50068", "#0067E6"))
xlim <- range(pca_results$x[, 1])
ylim <- range(pca_results$x[, 2])
par(mar = c(5, 4, 4, 6), xpd = TRUE)
plot(pca_results$x[, 1], pca_results$x[, 2],
     cex = 1.2, pch = 19, col = targets$Group,
     xlab = "PC1", ylab = "PC2",
     xlim = xlim, ylim = ylim,
     main = "PCA (Groups)")
text(pca_results$x[, 1], pca_results$x[, 2],
     labels = rownames(pca_results$x), cex = 0.4, pos = 3)
legend("right", inset = c(-0.05, 0),
       legend = levels(targets$Group),
       col = c(1, 2), pch = 19,
       cex = 0.9, pt.cex = 1.2, x.intersp = 1.2,
       box.lty = 0, bg = "white")
```



This PCA plot shows that the dataset variability, represented by principal components PC1 and PC2, is not clearly explained by the “group” parameter (CTRL vs DIS). Although CTRL samples tend to cluster in the left portion of the plot (negative PC1 values), this separation is not distinctive since the same region also contains 2 DIS samples, representing half of the entire DIS group (2 out of 4 visible samples). Along the PC2 axis, the distinction is even less evident, with both groups distributed across a wide range of values without a clear separation pattern.

PCA plot by Sex

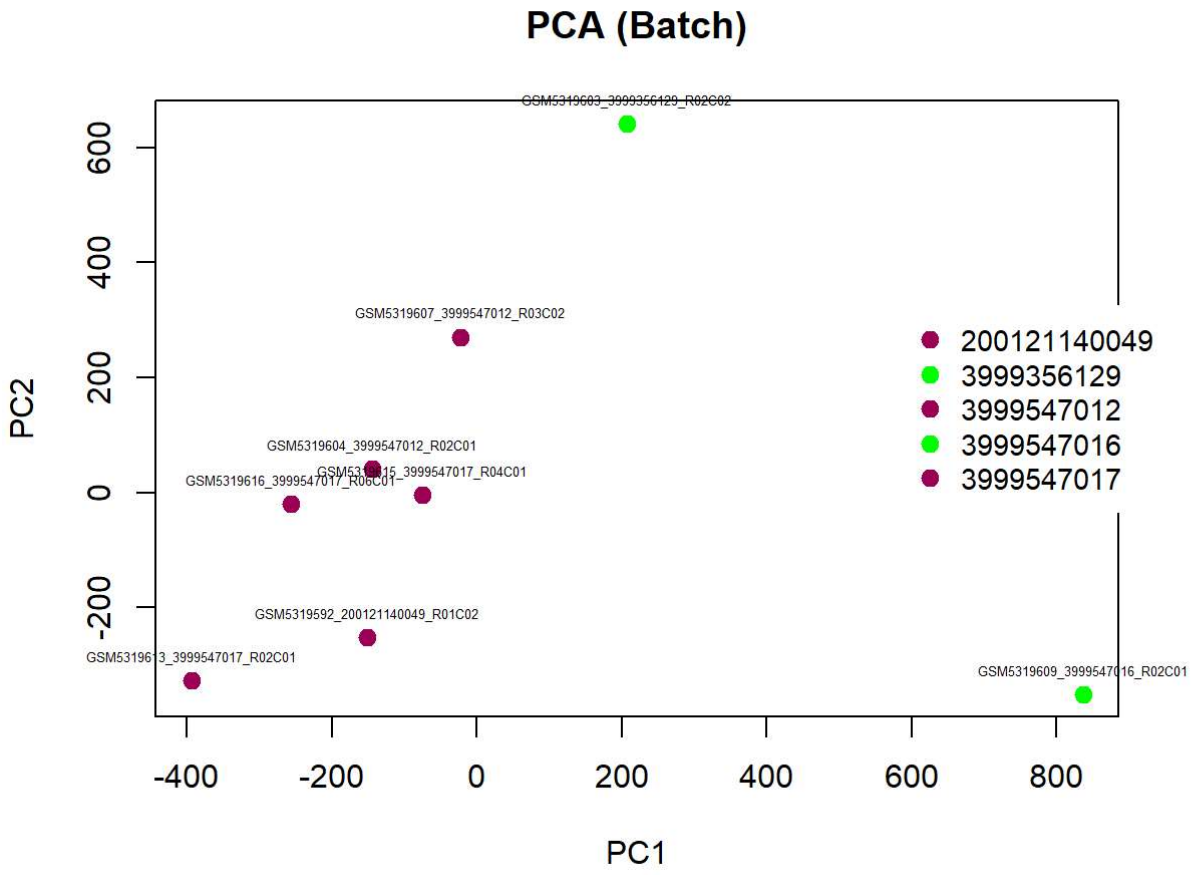
```
targets$Sex <- as.factor(targets$Sex)
palette(c("#C364CA", "#8BC4F9"))
par(mar = c(5, 4, 4, 6), xpd = TRUE)
plot(pca_results$x[, 1], pca_results$x[, 2],
     cex = 1.2, pch = 19, col = targets$Sex,
     xlab = "PC1", ylab = "PC2",
     xlim = xlim, ylim = ylim,
     main = "PCA (Sex)")
text(pca_results$x[, 1], pca_results$x[, 2],
     labels = rownames(pca_results$x), cex = 0.4, pos = 3)
legend("right", inset = c(-0.05, 0),
       legend = levels(targets$Sex),
       col = 1:nlevels(targets$Sex), pch = 19,
       cex = 0.9, pt.cex = 1.2, x.intersp = 1.2,
       box.lty = 0, bg = "white")
```



This PCA plot reveals that PC1 variability is not significantly explained by the SEX variable, as both male and female samples are distributed across the entire length of the graph along this axis. However, the situation is completely different for PC2, where a clear sex-based clustering pattern emerges: all female samples are grouped in the lower half of the plot , while male samples, with the exception of a single outlier, are clustered in the upper portion of the graph.

PCA plot by Batch (Sentrix_ID)

```
targets$Slide <- as.factor(targets$Slide)
palette(c("#9e0059", "green"))
par(mar = c(5, 4, 4, 6), xpd = TRUE)
plot(pca_results$x[, 1], pca_results$x[, 2],
     cex = 1.2, pch = 19, col = targets$Slide,
     xlab = "PC1", ylab = "PC2",
     xlim = xlim, ylim = ylim,
     main = "PCA (Batch)")
text(pca_results$x[, 1], pca_results$x[, 2],
     labels = rownames(pca_results$x), cex = 0.4, pos = 3)
legend("right", inset = c(-0.05, 0),
       legend = levels(targets$Slide),
       col = c(1, 2), pch = 19,
       cex = 0.9, pt.cex = 1.2, x.intersp = 1.2,
       box.lty = 0, bg = "white")
```



The samples show a strong batch effect. All samples from one batch (brown) cluster tightly on the left, while samples from other batches (green) appear as isolated and distant outliers. This indicates that Sentrix_ID contributes significantly to variance and that batch effects were not fully corrected by normalization.

9. Using the matrix of normalized beta values generated in step 7, identify differentially methylated probes between group CTRL and group DIS using the function assigned to each group.

A statistical comparison was performed on the normalized beta values. t-test was applied to each probe (corresponding to a CpG site) by iterating over the rows of the normalized beta value matrix, using the `t.test()` function in R.
The t-test allows comparison of group means while accounting for within-group variance, making it suitable for detecting site-specific differences.

```
My_t_test <- function(x) {
  t_test <- t.test(x ~ targets$Group)
  return(t_test$p.value)
}

p_values <- apply(beta_norm, 1, My_t_test)
```

Create a dataframe combining beta values with the raw p-values.

```
final_ttest <- data.frame(beta_norm, p_values_ttest = p_values)
```

Sort the probes by p-value in ascending order.

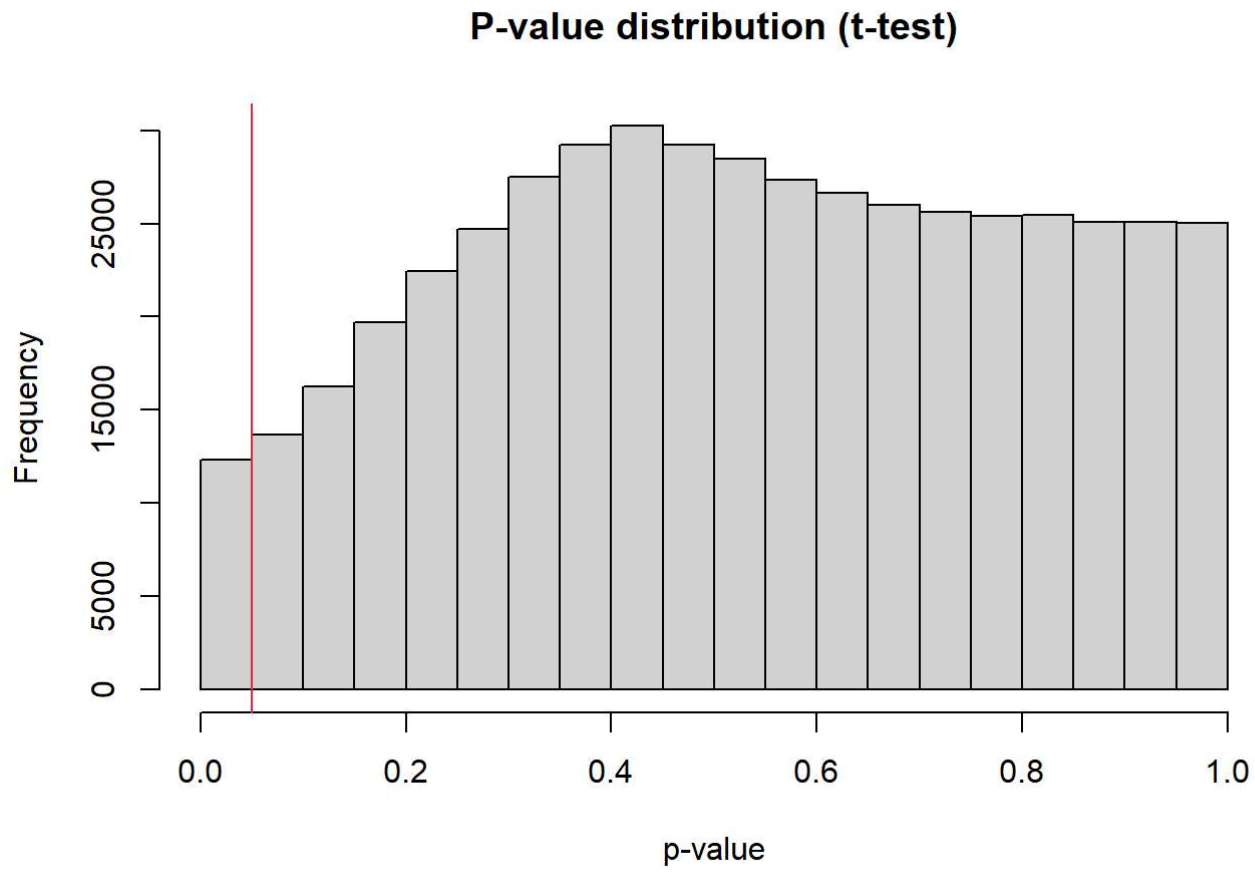
```
final_ttest <- final_ttest[order(final_ttest$p_values_ttest), ]
```

Filter significant probes with p-value <= 0.05.

```
final_ttest_th <- final_ttest[final_ttest$p_values_ttest <= 0.05, ]
```

Plot histogram of p-value distribution.

```
hist(final_ttest$p_values_ttest, main = "P-value distribution (t-test)",
     xlab = 'p-value')
abline(v = 0.05, col = "#ef233c")
```



The distribution is approximately uniform, with a slight increase toward higher p-values, indicating no strong overall signal of differential methylation across most probes. The red vertical line marks the significance threshold at 0.05. Despite the lower density near zero, the number of probes with p-values below 0.05 still

represents about 1% of the total. The total number of significant probes was reported before applying multiple testing correction.

Check the number of differentially methylated probes before correction.

```
dim(final_ttest_th)
```

```
## [1] 12351     9
```

10. Apply multiple test correction and set a significant threshold of 0.05. How many probes do you identify as differentially methylated considering nominal pValues? How many after Bonferroni correction? How many after BH correction?

When multiple tests are performed simoultaneously, the likelihood of obtaining type 1 errors increases. This problem is mitigated using two different approaches:
- *Benjamini Hochberg* correction to control FDR (False Discovery Rate), which correspond to the expected proportion of type 1 errors among significant results;
- *Bonferroni correction* to control FWER (Family-Wise Error Rate), which correspond to the probability of obtaining type 1 errors among all the hypothesis.

To apply these two types of correction, we will use `p.adjust()` function. The output is stored in the dataframe.

```
corr_pval_BH <- p.adjust(final_ttest$p_values_ttest, method = "BH")  # Benjamini-Hochberg
corr_pval_bonferroni <- p.adjust(final_ttest$p_values_ttest, method = "bonferroni")
```

Combine the corrected p-values into the final dataframe for downstream filtering and visualization.

```
final_ttest_corr <- data.frame(final_ttest, corr_pval_BH, corr_pval_bonferroni)
```

Count the number of differentially methylated probes before and after correction.

```
before_correction <- nrow(final_ttest_corr[final_ttest_corr$p_values_ttest <= 0.05, ])
after_Bonferroni <- nrow(final_ttest_corr[final_ttest_corr$corr_pval_bonferroni <= 0.05, ])
after_BH <- nrow(final_ttest_corr[final_ttest_corr$corr_pval_BH <= 0.05, ])
```

Create a summary dataframe with all the results.

```
diff_meth_df <- data.frame(before_correction, after_Bonferroni, after_BH)
rownames(diff_meth_df) <- c("differentially methylated probes")
```

Print the summary dataframe.

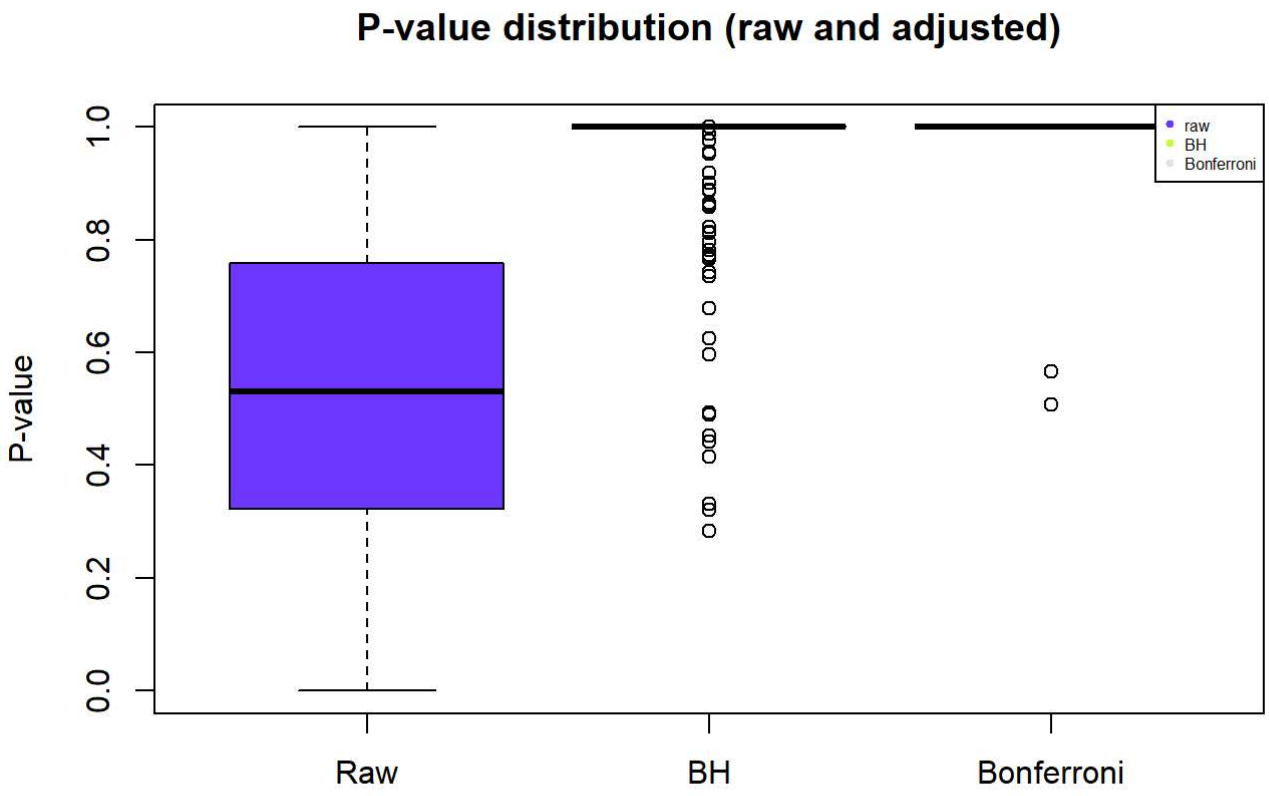
```
knitr::kable(diff_meth_df)
```

	before_correction	after_Bonferroni	after_BH
differentially methylated probes	12351	0	0

```
#reset plotting area
par(mfrow = c(1,1))
```

Display boxplots of raw and adjusted p-values.

```
boxplot(final_ttest_corr[,9:11],
        col = c("#7038FF", "#C7FF38", "seashell2"),
        names = c("Raw", "BH", "Bonferroni"),
        main = 'P-value distribution (raw and adjusted)',
        ylab = "P-value")

legend("topright",
       legend = c("raw", "BH", "Bonferroni"),
       col = c("#7038FF", "#C7FF38", "seashell2"),
       pch = 19, cex = 0.5, xpd = TRUE)
```



The box plot shows the distribution of p-values before and after correction. In this case, after applying both Bonferroni and BH corrections, no probes remained significant.

11. Produce a volcano plot and a Manhattan plot of the results of differential methylation analysis

Volcano plot

The volcano plot illustrates the outcome of the differential methylation analysis between diseased (DIS) and control (CTRL) groups. Each point in the plot corresponds to a CpG site tested for a significant difference in methylation (*DMPs*).
The x-axis shows the difference in average beta values between the two groups, while the y-axis represents the statistical significance of that difference as –log10(p-value). CpG sites showing both high statistical significance and strong methylation differences are highlighted by color, especially those surpassing the Benjamini-Hochberg adjusted p-value threshold (shown in pink).

Extract beta values (columns 1 to 8).

```
beta <- final_ttest_corr[, 1:8]
```

Calculate mean beta-values for each group.

```
beta_groupCTRL <- beta[, targets$Group == "CTRL"]
mean_beta_groupCTRL <- apply(beta_groupCTRL, 1, mean)

beta_groupDIS <- beta[, targets$Group == "DIS"]
mean_beta_groupDIS <- apply(beta_groupDIS, 1, mean)
```

Calculate delta (DIS - CTRL).

```
delta_average <- mean_beta_groupDIS - mean_beta_groupCTRL
```

Identify probes significant after BH correction.

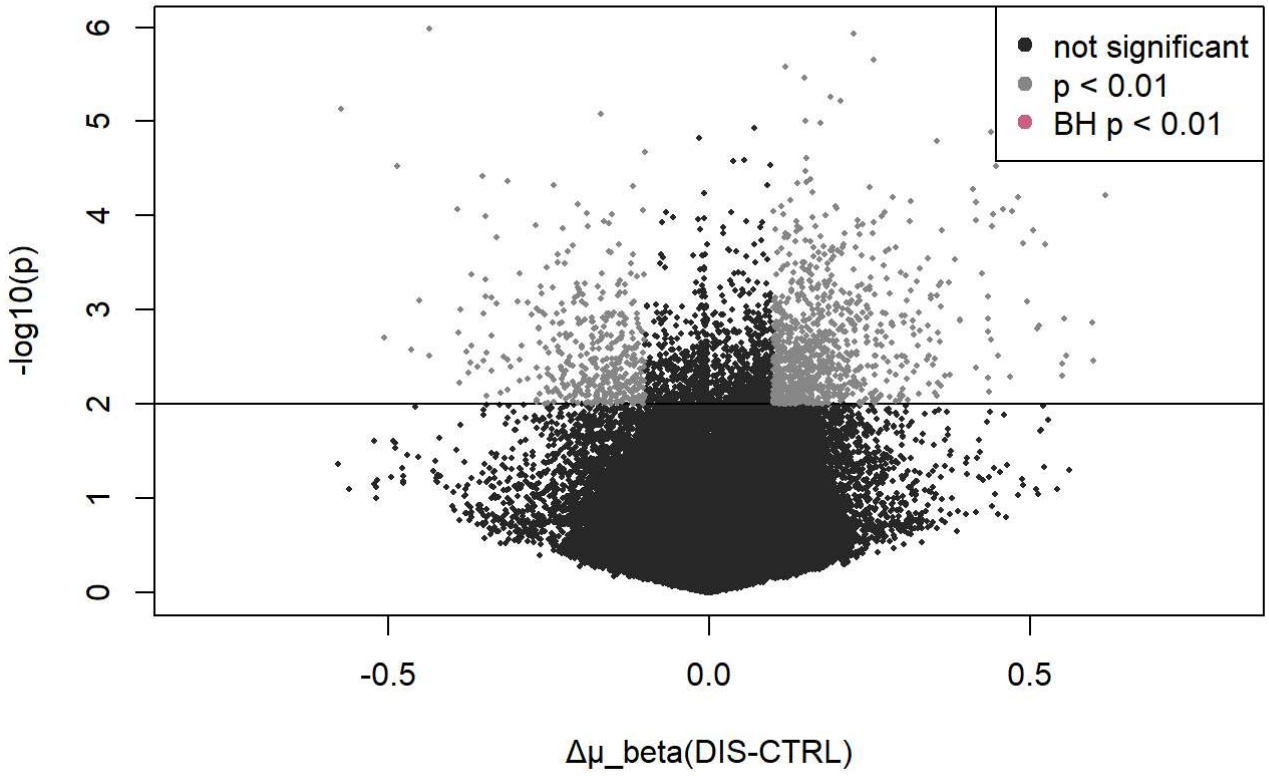
```
BH_sig <- final_ttest_corr$corr_pval_BH < 0.01
```

Create a dataframe for the volcano plot.

```
toVolcPlot <- data.frame(
  delta_average,
  minus_log10_p_val = -log10(final_ttest_corr$p_values_ttest),
  BH_sig
)
```

Display the volcano plot.

```
plot(toVolcPlot[,1], toVolcPlot[,2],
     pch=16, cex=0.4, col='grey16',
     xlab='Δμ_beta(DIS-CTRL)', ylab='-log10(p)',
     xlim=c(-0.8,0.8))
abline(h = -log10(0.01))

nominal_sig <- toVolcPlot[
  abs(toVolcPlot[,1]) > 0.1 &
    toVolcPlot[,2] > -log10(0.01), ]
points(nominal_sig[,1], nominal_sig[,2], pch=16, cex=0.4, col="snow4")

BH_sig_points <- toVolcPlot[
  abs(toVolcPlot[,1]) > 0.1 &
    toVolcPlot$BH_sig, ]
points(BH_sig_points[,1], BH_sig_points[,2], pch=19, cex=0.6, col="hotpink3")

legend("topright",
       legend=c("not significant", "p < 0.01", "BH p < 0.01"),
       col=c("grey16", "snow4", "hotpink3"), pch = 19)
```



In this specific plot, no CpG sites reach the adjusted significance level, and thus no pink points are visible. This indicates that, although some sites show moderate differences, none pass the threshold for statistical significance after correction.

Manhattan Plot

In the Manhattan plot, the results of the same analysis are displayed across the genome. Each dot represents a CpG site, plotted according to its chromosomal position on the x-axis and its –log10(p-value) on the y-axis. This genome-wide overview enables the identification of differentially methylated regions (DMRs), highlighting clusters of epigenetic alterations that may play a role in disease processes.

Add probe IDs as a column.

```
final_ttest_corr_anno <- data.frame(rownames(final_ttest_corr), final_ttest_corr)
colnames(final_ttest_corr_anno)[1] <- "IlmnID"
```

Merge with Illumina manifest file (must contain IlmnID, CHR, MAPINFO).

```
final_ttest_corr_anno <- merge(final_ttest_corr_anno, Illumina450Manifest_clean, by = "IlmnI
D")
```

Prepare the dataframe for `qqman` library.

```
input_Manhattan <- data.frame(
  ID = final_ttest_corr_anno$IlmnID,
  CHR = final_ttest_corr_anno$CHR,
  MAPINFO = final_ttest_corr_anno$MAPINFO,
  PVAL = final_ttest_corr_anno$p_values_ttest
)
```

Convert chromosomes X and Y to numeric (23 and 24).

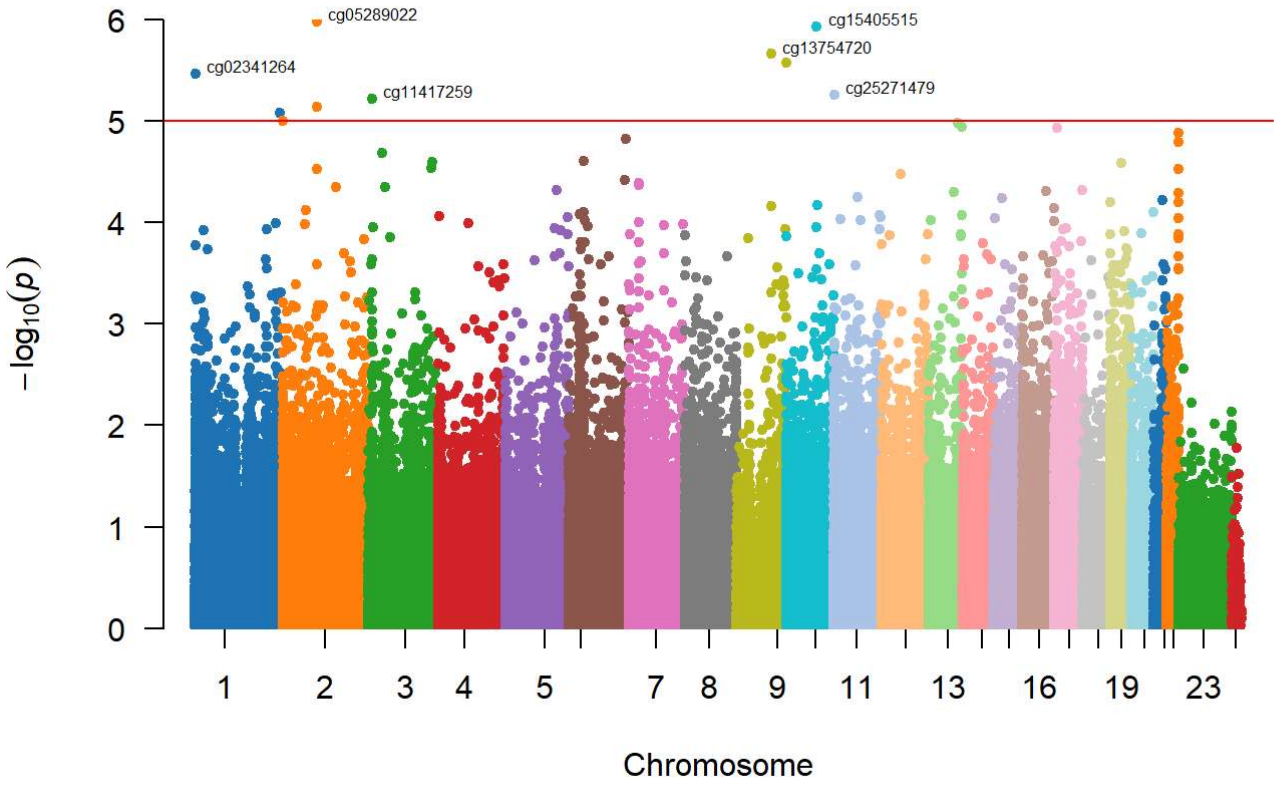
```
levels(input_Manhattan$CHR)[levels(input_Manhattan$CHR) == "X"] <- "23"
levels(input_Manhattan$CHR)[levels(input_Manhattan$CHR) == "Y"] <- "24"
input_Manhattan$CHR <- as.numeric(as.character(input_Manhattan$CHR))
```

Load the `qqman` library and define a color palette
qqman provides functions to generate Manhattan and QQ plots. It is useful for visualizing p-values in GWAS or methylation studies to detect significant loci.

```
install.packages("qqman")
library(qqman)
color_palette <- c(
  "#1F77B4", "#FF7F0E", "#2CA02C", "#D62728", "#9467BD", "#8C564B", "#E377C2",
  "#7F7F7F", "#BCBD22", "#17BECF", "#AEC7E8", "#FFBB78", "#98DF8A", "#FF9896",
  "#C5B0D5", "#C49C94", "#F7B6D2", "#C7C7C7", "#DBDB8D", "#9EDAE5",
  "#1F77B4", "#FF7F0E", "#2CA02C", "#D62728"
)
```

Display the Manhattan plot.

```
manhattan(
  input_Manhattan,
  snp = "ID",
  chr = "CHR",
  bp = "MAPINFO",
  p = "PVAL",
  annotatePval = 0.00001,
  col = color_palette,
  suggestiveline = FALSE,
  genomewideline = -log10(0.00001)
)
```




```
#reset plotting layout
par(mfrow = c(1, 1))
```

CpG sites above the significance threshold (red line) are considered statistically significant after correction. Several top hits show strong association signals and may represent biologically relevant DMPs. The alternating colors per chromosome enhance readability and highlight the genomic distribution of significant probes.

12. Produce an heatmap of the top 100 significant, differentially methylated probes.

The *heatmap* shows the methylation levels of selected CpG sites across all samples included in the study. Each row corresponds to a CpG site, and each column to a sample, with the color scale ranging from green (low methylation) to pink (high methylation). The samples are grouped by hierarchical clustering, revealing clear patterns of methylation that differentiate the CTRL and DIS groups. This clustering helps confirm the presence of consistent methylation signatures and supports the discriminative power of the selected sites. In this project, different clustering methods were applied: *single*, *complete*, and *average linkage*. These methods differ in how they define the distance between clusters.

Load required library

`gplots` is an R package that provides enhanced plotting functions for data visualization, including improved heatmaps, dendrograms, and tools for exploratory analysis of high-dimensional data.

```
if (!require("gplots")) install.packages("gplots")
library(gplots)
```

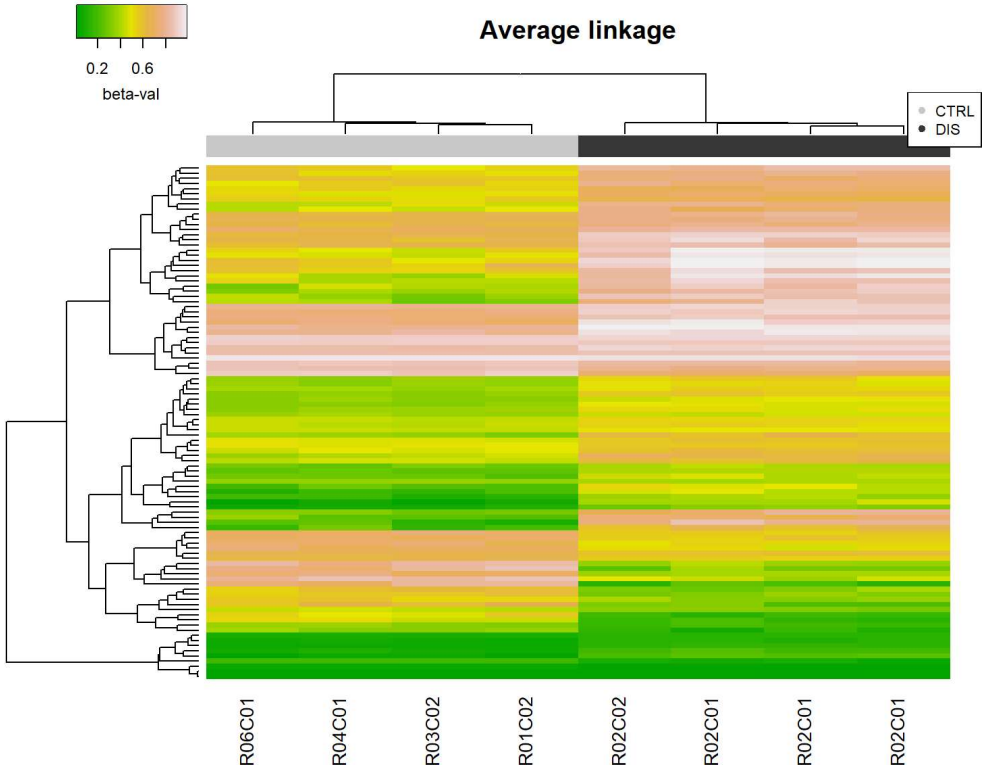
Prepare the input matrix for the heatmap.

```
input_heatmap <- as.matrix(final_ttest[1:100, 1:8])
```

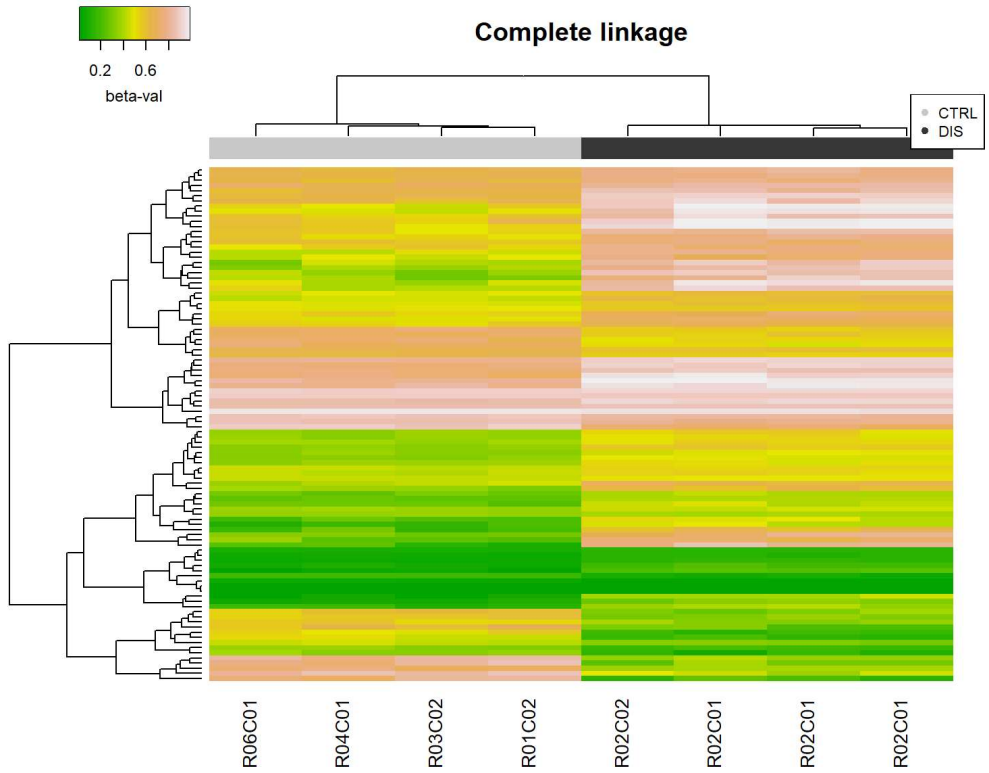
Assign colors to groups.

```
group_color <- c()
for (i in 1:ncol(input_heatmap)) {
  name <- colnames(input_heatmap)[i]
  sample_id <- strsplit(name, "_")[[1]][1]
  matching_row <- which(targets$SampleID == sample_id)
  if (length(matching_row) > 0) {
    if (targets$Group[matching_row] == "CTRL") {
      group_color <- c(group_color, "#CBCBCB")
    } else {
      group_color <- c(group_color, "#393939")
    }
  }
}
```

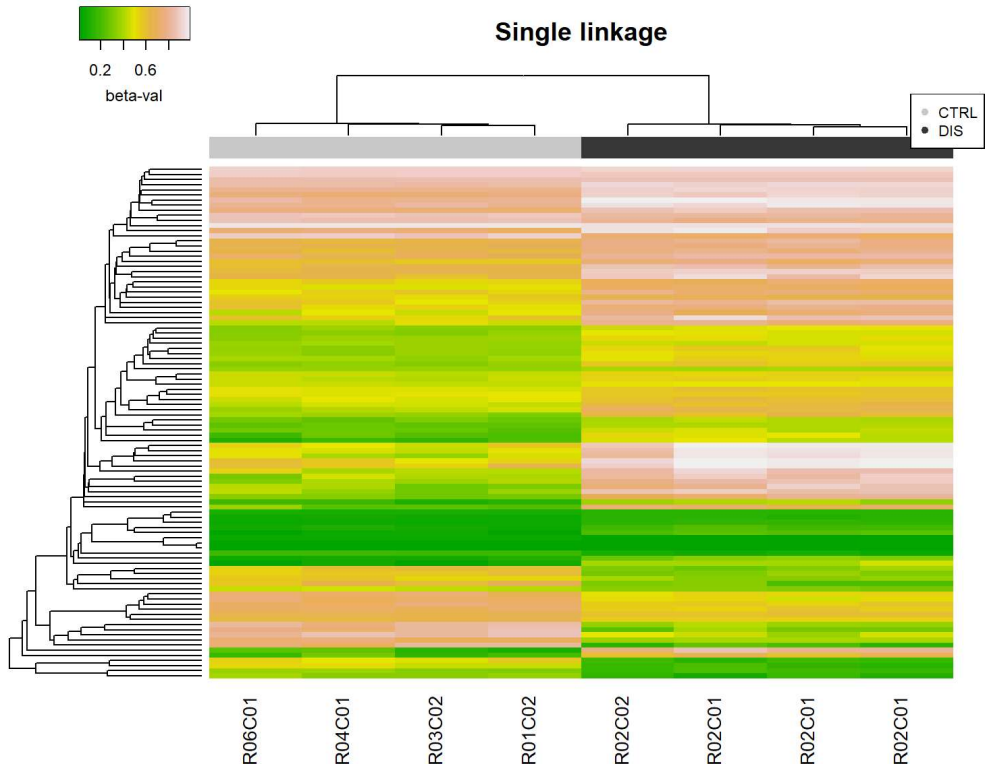
Display the Average linkage heatmap.



Display the Complete linkage heatmap.



Display the Single linkage heatmap.



The three hierarchical clustering methods (Single, Complete, and Average linkage) show consistent global separation between CTRL and DIS samples, indicating robust methylation differences. Complete and Average linkage provide more compact and biologically interpretable clusters, while Single linkage shows a chaining effect with elongated branches, which may reduce cluster resolution.
Overall, Average linkage appears to balance sensitivity and interpretability, making it a suitable choice for downstream analysis.